

Dossier de Projet - Application Mini CRM SaaS

Document de conception et d'architecture rédigé en vue du passage du titre professionnel Concepteur Développeur d'Applications (CDA).

1. Introduction et Contexte du Projet

1.1. Contexte métier et problématique

La gestion administrative est souvent le point faible des artisans, freelances et très petites entreprises (TPE). La plupart de ces acteurs utilisent encore des solutions bureautiques inadaptées (Excel, Word) pour la création de leurs devis et le suivi de leur facturation. Cette approche présente de nombreux défauts :

- Perte de temps liée à la double saisie.
- Risque d'erreur de calcul (notamment sur la TVA).
- Manque de traçabilité des relances clients.
- Perte de documents.

Le marché propose des ERP (Enterprise Resource Planning) ou des CRM très complets (Salesforce, Odoo), mais ces solutions s'avèrent souvent trop complexes, surdimensionnées et trop coûteuses pour des indépendants.

1.2. La proposition de valeur (Objectif du projet)

L'objectif de ce projet est de concevoir et développer une application web de type SaaS (Software as a Service), appelée "Mini CRM". Cette application se veut :

- **Simple et intuitive** : Pas de fonctionnalités superflues, on se concentre sur le cœur de métier (Clients, Devis, Factures).
- **Accessible partout** : Une solution web responsive (PC, tablette, mobile).
- **Automatisée** : Transformation d'un devis en facture en un clic, génération de PDF automatique, calcul des taxes instantané.

1.3. Les Personas

Persona Principal : "Julien, Artisan Plombier"

- *Besoins* : Gérer son carnet d'adresses, faire des devis sur les chantiers depuis sa tablette, éditer des factures le soir chez lui.
 - *Frustrations* : Perd trop de temps le soir à faire ses factures sur Word, oublie parfois de relancer des clients qui n'ont pas payé.
-

2. Spécifications Fonctionnelles et Besoins (Agilité)

Le développement a été piloté par les besoins utilisateurs à travers des User Stories (US) gérées dans un tableau Kanban. Chaque US est soumise à des critères d'acceptation stricts.

2.1. Epic 1 : Gestion des accès

- **US-01 - Inscription** : En tant que visiteur, je veux pouvoir créer un compte avec mon email et un mot de passe sécurisé.
 - *Critère d'acceptation* : Le mot de passe doit être hashé. L'email doit être unique.

- **US-02 - Authentification** : En tant qu'utilisateur, je veux me connecter à mon espace.
 - *Critère d'acceptation* : Génération d'un token JWT sécurisé pour maintenir la session.

2.2. Epic 2 : Gestion du portefeuille client

- **US-03 - Création Client** : En tant qu'utilisateur, je veux ajouter un client (Nom, Entreprise, Email, Téléphone, Adresse).
- **US-04 - Liste Clients** : En tant qu'utilisateur, je veux voir la liste de mes clients et pouvoir la filtrer/rechercher.

2.3. Epic 3 : Le tunnel de facturation

- **US-05 - Création de Devis** : En tant qu'utilisateur, je veux créer un devis, l'associer à un de mes clients, et y ajouter de multiples lignes de prestations.
 - *Critère d'acceptation* : Les calculs (Total ligne, Total HT, Total TTC) doivent s'effectuer en temps réel côté client et être revérifiés côté serveur.
 - **US-06 - Cycle de vie du Devis** : Je veux pouvoir changer le statut du devis (Brouillon, Envoyé, Accepté, Refusé).
 - **US-07 - Conversion en Facture** : Je veux qu'un devis "Accepté" puisse être transformé en facture.
 - *Critère d'acceptation* : Le système doit générer un numéro de facture unique (ex: FACT-2026-001) incrémental. Les lignes du devis doivent être copiées sans altérer le devis original (principe d'immuabilité financière).
 - **US-08 - Export PDF** : Je veux générer et télécharger la facture au format PDF.
-

3. Conception UI/UX et Maquettage

L'interface de l'application a été pensée en *Mobile-First*.

3.1. Charte graphique

- **Couleurs** : Utilisation d'un thème clair axé sur la productivité (Bleu professionnel pour les actions primaires, blanc et gris pour les fonds afin de faire ressortir la data).
- **Typographie** : Utilisation de polices sans-serif lisibles (ex: Inter / Roboto).

3.2. Parcours Utilisateur (User Flow)

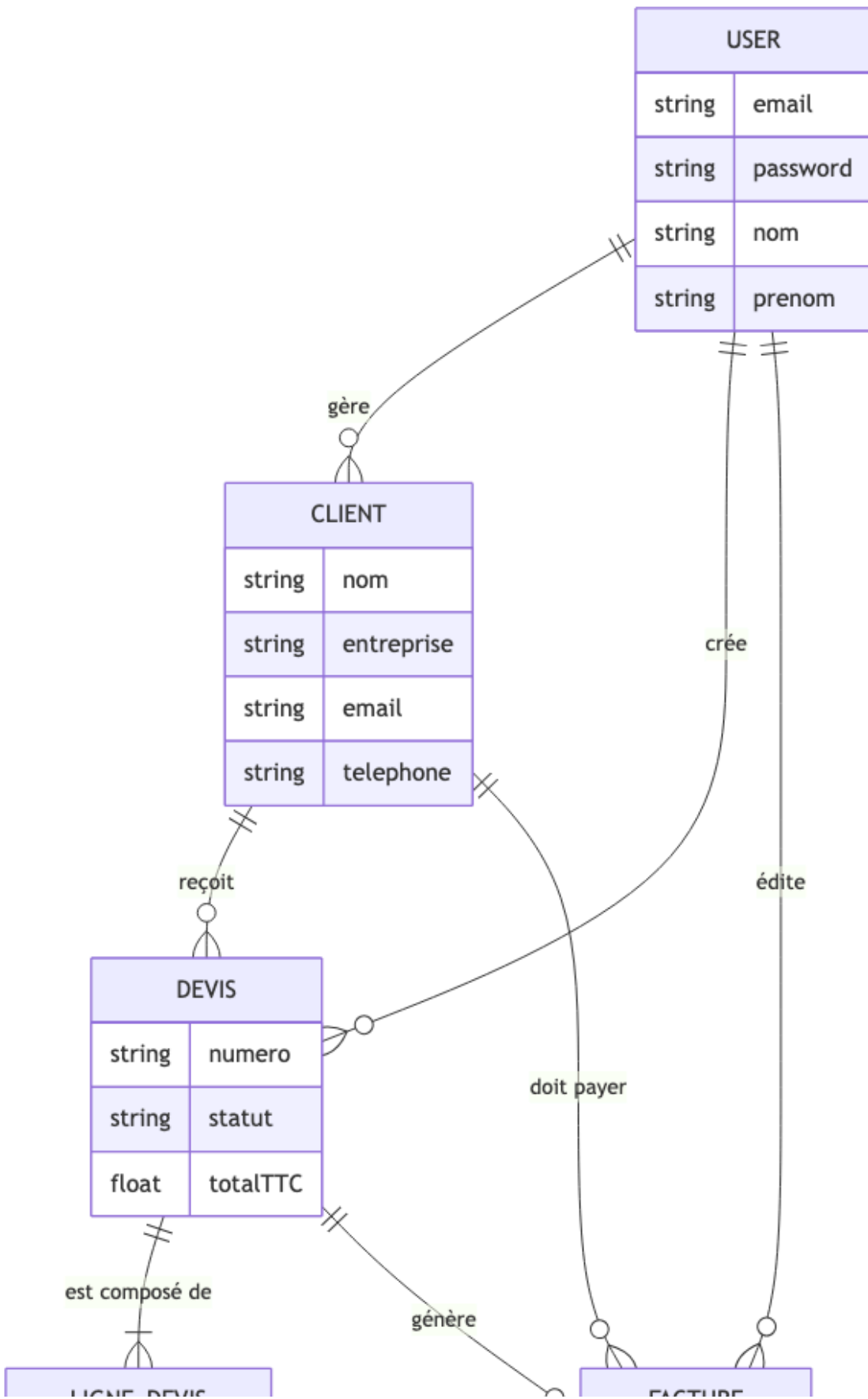
1. L'utilisateur arrive sur la *Landing Page* (Connexion).
 2. Après connexion, il atterrit sur le *Dashboard* présentant un résumé de son activité (Chiffre d'affaires, factures en attente).
 3. Une barre de navigation latérale (Sidebar) lui permet d'accéder aux vues : Clients, Devis, Factures.
 4. Les vues principales se composent de tableaux de données (DataTables) avec des boutons d'actions (Créer, Voir, Modifier, Supprimer).
-

4. Conception des Données (UML et BDD)

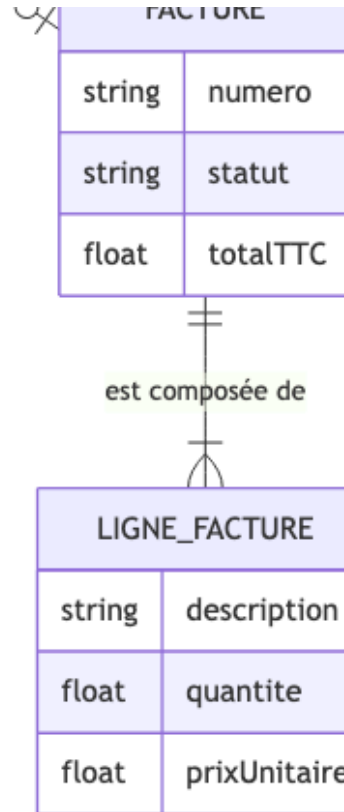
La persistance des données repose sur une base relationnelle (SGBDR) PostgreSQL.

4.1. Modèle Conceptuel des Données (MCD)

Le MCD met en évidence la relation de "Propriété" : chaque ressource de l'application (Client, Devis, Facture) est liée à l'Utilisateur qui l'a créée, garantissant un cloisonnement total des données (Multi-tenant).



LIGNE_DEVIS	
string	description
float	quantite
float	prixUnitaire



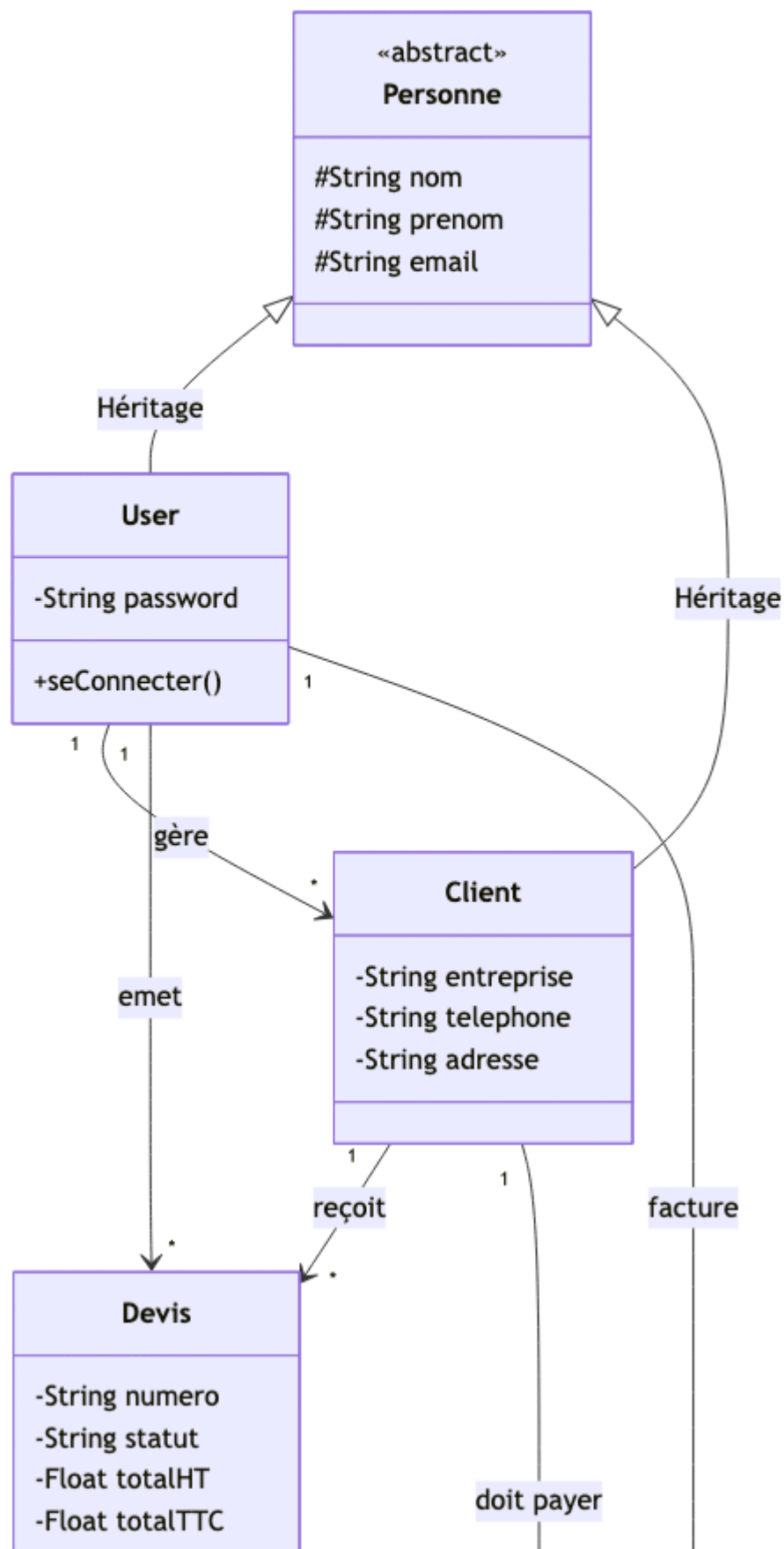
4.2. Dictionnaire de données détaillé

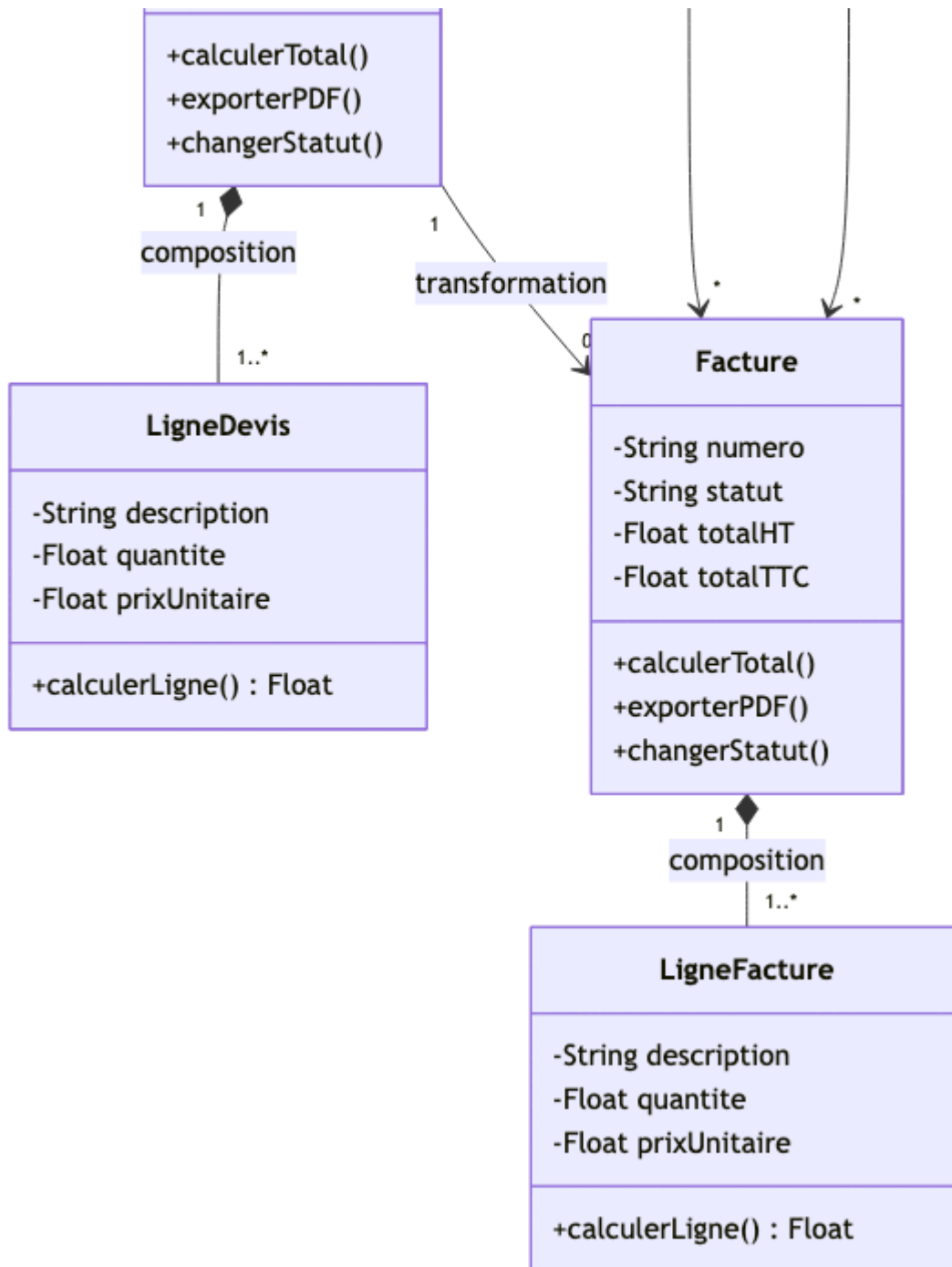
Entité	Attribut	Type	Contraintes	Description
User	id	INT	PK, Auto	Identifiant unique
	email	VARCHAR	UNIQUE, NOT NULL	Identifiant de connexion
	password	VARCHAR	NOT NULL	Mot de passe hashé
	nom, prenom	VARCHAR	NOT NULL	Identité de l'utilisateur
Client	id	INT	PK, Auto	Identifiant unique
	userId	INT	FK, NOT NULL	Référence à l'utilisateur propriétaire
	nom, prenom	VARCHAR	NOT NULL	Contact du client
	entreprise	VARCHAR	NULL	Nom de la société
	email, tel	VARCHAR	NULL	Coordonnées
Devis	id	INT	PK, Auto	Identifiant unique

	numero	VARCHAR	UNIQUE, NOT NULL	Référence (ex: DEV-2026-001)
	statut	VARCHAR	NOT NULL	Brouillon, Envoyé, Accepté, Refusé
	totalTTC	FLOAT	NOT NULL	Montant final calculé
	clientId	INT	FK, NOT NULL	Le client facturé
Facture	numero	VARCHAR	UNIQUE, NOT NULL	Référence légale de facture
	devisId	INT	FK, UNIQUE	Le devis à l'origine de la facture (optionnel)

4.3. Modèle Logique et Classes métiers

L'interaction entre les différentes entités métiers au sein du code est représentée par le diagramme de classes suivant :





5. Architecture Logique et Choix Technologiques

L'application respecte l'architecture **Client-Serveur** avec une séparation stricte des préoccupations (Séparation of Concerns).

5.1. Le Backend : Node.js & Express.js

Le serveur Backend expose une **API RESTful**. Le choix de Node.js se justifie par sa performance sur les opérations I/O (requêtes base de données, génération PDF asynchrone) et sa cohérence avec l'écosystème JavaScript du Frontend.

- **ORM : Prisma.** Utilisé pour interagir avec PostgreSQL. Prisma offre une sécurité au niveau du typage (Type-safety), empêche les injections SQL, et gère le versioning de la base de données via `Prisma Migrate`.
- **Génération PDF : Puppeteer.** Un navigateur Chrome "Headless" est exécuté côté serveur. Il convertit une page HTML dynamique contenant les données de la facture en un document PDF A4 prêt à l'emploi.

5.2. Le Frontend : React.js & Vite

L'interface est une **Single Page Application (SPA)** développée avec React.

- **Vite :** Outil de build nouvelle génération, choisi pour sa rapidité de Hot Module Replacement (HMR) comparé à Webpack.
- **Tailwind CSS :** Framework CSS utilitaire permettant de concevoir des interfaces sur mesure directement dans le JSX sans multiplier les fichiers CSS.

- **Axios** : Client HTTP utilisé pour consommer l'API, avec une configuration d'intercepteurs pour attacher automatiquement le token de sécurité.
-

6. Politique de Sécurité

La sécurité des données (particulièrement financières) est un enjeu critique.

6.1. Authentification et Autorisation

- **JWT (JSON Web Token)** : Lors du login, le backend génère un token signé avec une clé secrète (`JWT_SECRET`). Ce token contient l' `id` de l'utilisateur. Chaque route protégée de l'API possède un Middleware qui vérifie la validité et la signature du token.
- **Cloisonnement des données** : Le backend s'assure systématiquement que l'utilisateur qui fait la requête est bien le propriétaire de la ressource. (Ex: un User A ne peut pas lire le Devis du User B, même s'il en devine l'ID).

6.2. Protections contre les vulnérabilités OWASP

- **XSS (Cross-Site Scripting)** : L'utilisation de React empêche nativement les failles XSS, car React échappe automatiquement les variables lors de la manipulation du DOM virtuel.
 - **Injections SQL** : L'utilisation de l'ORM Prisma neutralise toutes les requêtes directes en construisant des requêtes préparées sécurisées.
 - **Stockage des mots de passe** : Les mots de passe ne sont jamais stockés en clair. Ils sont salés et hashés à l'aide de l'algorithme `bcrypt` .
-

7. Qualité Logicielle : Tests et CI/CD

La fiabilité de l'application est garantie par des processus d'assurance qualité stricts.

7.1. Stratégie de Tests (Jest)

Le cœur métier de l'application (les calculs de devis et la sécurité de l'authentification) est couvert par des tests unitaires automatisés utilisant **Jest**.

- **Tests d'authentification** : Validation de la création de compte, du hashage correct du mot de passe, de la gestion des erreurs (email existant), et de la génération du token JWT.
- **Tests métiers (Calculs)** : Vérification stricte des algorithmes de calcul financier (`calculDevis.test.js`) pour s'assurer que les totaux HT, les taxes (TVA), et les totaux TTC sont mathématiquement exacts, évitant ainsi toute anomalie de facturation en production.

7.2. Intégration Continue (GitHub Actions)

L'intégration continue (CI) est gérée de manière automatisée grâce à **GitHub Actions**. Un pipeline (workflow `main.yml`) est déclenché à chaque "Push" ou "Pull Request" sur la branche principale :

- Il installe l'environnement Node.js.
 - Il télécharge les dépendances.
 - Il exécute la suite de tests unitaires Jest. Ce mécanisme empêche tout déploiement de code cassé (régressions) sur le serveur de production.
-

8. Infrastructure et Déploiement Cloud (CD)

Le projet a été déployé en production dans un environnement cloud professionnel chez **OVHcloud**, en appliquant les bonnes pratiques DevOps.

7.1. Conteneurisation (Docker)

L'architecture est entièrement conteneurisée pour garantir la portabilité et éviter le problème du "ça marche sur ma machine". Le fichier `docker-compose.yml` orchestre 5 services simultanément :

1. **db** : Le moteur PostgreSQL.
2. **redis** : Un cache en mémoire pour accélérer certaines requêtes.
3. **backend** : L'API Node.js.
4. **frontend** : L'application React compilée par Nginx/Vite.
5. **caddy** : Le serveur web frontal.

7.2. Reverse Proxy et Sécurité Réseau (Caddy Server)

Caddy a été choisi comme serveur web principal et reverse-proxy. Il agit comme l'unique point d'entrée du serveur :

- Il écoute sur le domaine `m-atici.fr`.
- Il redirige le trafic `/api/*` vers le conteneur backend.
- Il sert les fichiers statiques du frontend pour le reste des requêtes.
- **Avantage majeur** : Caddy génère et renouvelle automatiquement les certificats SSL/TLS via *Let's Encrypt*. L'application est donc sécurisée en HTTPS sans configuration complexe.

7.3. Monitoring et Statistiques

Un outil d'analyse de logs, **GoAccess**, est branché directement sur les logs de Caddy. Il permet de visualiser le trafic réseau en temps réel via un tableau de bord sécurisé, permettant de surveiller la charge du serveur et de repérer les éventuelles attaques (DDoS, scans de ports).

7.4. Script d'automatisation

Un script Bash (`deploy-staging.sh`) a été écrit pour faciliter les mises à jour. Il exécute de manière séquentielle :

1. La récupération du dernier code via `git pull`.
2. La reconstruction des images Docker modifiées (`docker-compose build`).
3. Le redémarrage sans interruption des conteneurs.
4. L'exécution automatique des migrations de base de données Prisma en production (`npx prisma migrate deploy`).

9. Bilan et Perspectives

Le développement de ce Mini CRM a permis de valider l'ensemble des compétences attendues par le titre de Concepteur Développeur d'Applications. Le produit actuel constitue un **MVP (Minimum Viable Product)** fonctionnel et déployé en production.

Perspectives d'évolution techniques et fonctionnelles :

- Ajout d'un système de relance client automatique par email (via Nodemailer et une tâche Cron).
- Intégration d'une solution de paiement en ligne (Stripe) directement sur le lien de la facture PDF.
- Mise en place de tests unitaires systématiques sur le backend avec Jest pour augmenter la couverture de code avant les futures mises à jour.